

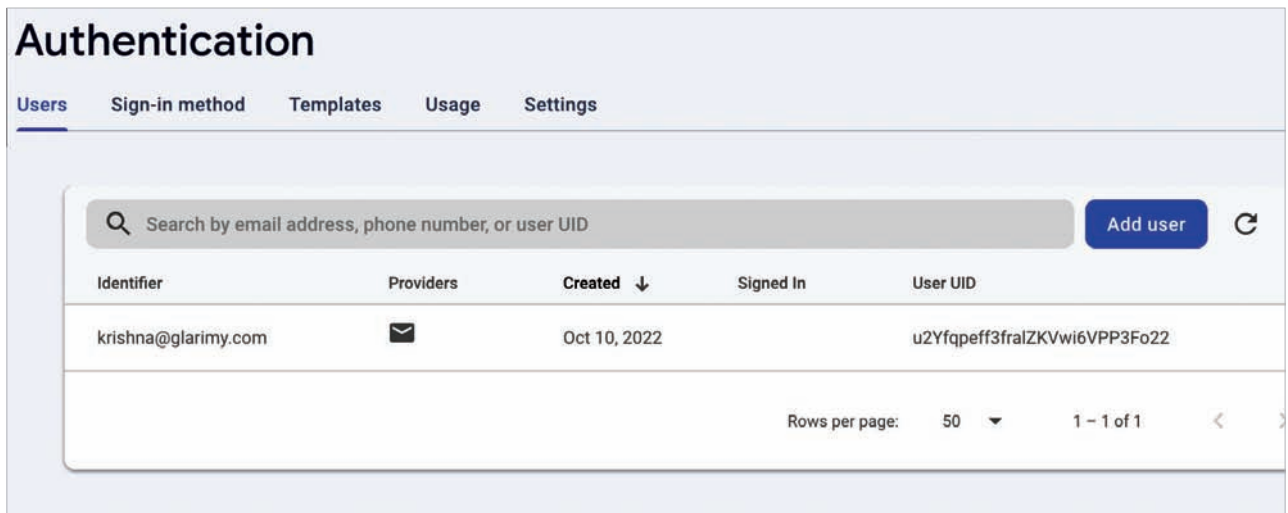
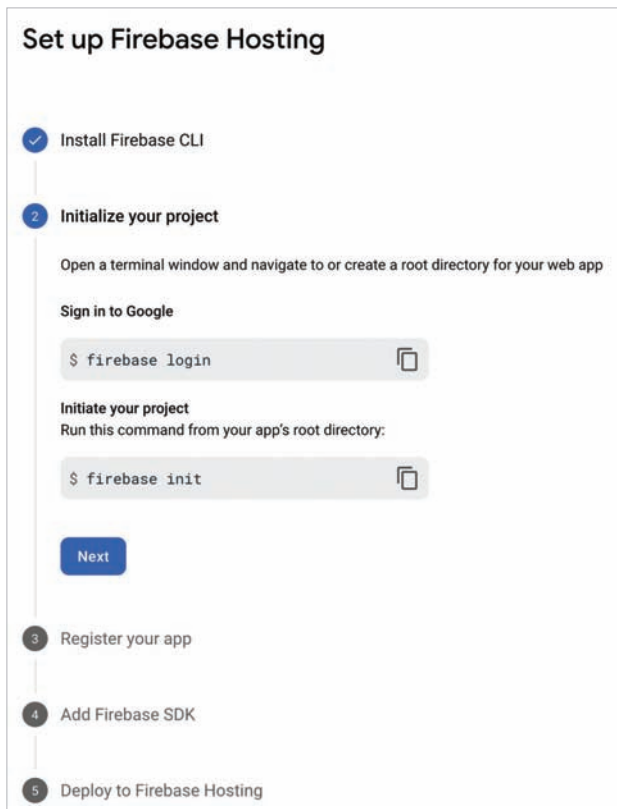


Figure 3: Authentication setup

Figure 4: Firebase *Hosting* setup

Now, create a collection with the name *journal-entries*. Create a dummy document with an auto-generated ID along with the structure, as shown in Figure 2. The document consists of three fields, namely, author (string), date (number to store the time in UNIX epoch format) and description (string).

All the journal entries will be stored in this *journal-entries* collection at runtime.

3. Set up authentication

We intend to control access to the application via a login. For that, we need to configure authentication.

Click on *Authentication* under the *Build* category of the *Project Review* pane. It opens the *Authentication* page. Click on *Get Started*. Choose and enable email/password as the sign-in provider. In other words, users must provide an email-id and password to sign in to the web app.

We want to restrict the app to a few predefined users. So, add new users under the *User* tab, as shown in Figure 3. We want only the users in this list to be able to sign in to the application.

Do not worry about the privacy of the passwords. Once the admin enters the details, no one (including the admin) can view the password again in a readable format.

4. Develop the client

Click on *Hosting* under the *Build* category of the *Project Review* pane. It opens the *Hosting* page. Click on *Get Started*. The wizard looks as shown in Figure 4. Follow the instructions to install Firebase SDK on your machine and develop the client application.

At Step 4 of the wizard, make a note of the generated *firebaseConfig*. In our case, it looks somewhat like the following:

```
const firebaseConfig = {
  apiKey: "AIzaSyXXXXXXXXXXXXXwwvZgtbrnLiqKFPE",
  authDomain: "glarimy-firebase.firebaseio.com",
  projectId: "glarimy-firebase",
  storageBucket: "glarimy-firebase.appspot.com",
  messagingSenderId: "7822123479139",
  appId: "1:782281879139:web:aad87vdcdeee3914d256",
  measurementId: "G-F86URYCS0X"
};
```